Paris Dauphine University x ENS x Mines Paris – PSL

Data Lab A1 project report

Collaborative Filtering by Similarity Fusion

Professor: Alexandre Verine

Group: 1p2v

Félix de Batz
Léon Hogenmuller
Thanina Siouani

october 2026

Contents

1 Baseline paper : Collaborative Filtering by Similarity Fusion **2**
- 1.1 Problem and Motivation . . . 2
- 1.2 Normalisation and Individual Predictors . . . 2
- 1.3 Probabilistic Fusion Framework . . . 2
- 1.4 Probability Estimation and Combined Similarity . . . 2
- 1.5 Unified Weights and Final Prediction . . . 3

2 Our take **3**
- 2.1 Integrating the movie features into our model . . . 3
- 2.2 Computing the neighborhood . . . 3
- 2.3 Computing the similarity . . . 4
- 2.4 Computing the similarity inside the rectangle . . . 4
- 2.5 Adding regularization . . . 4
- 2.6 Predicting a missing rating . . . 4

3 Experimental setup **5**

4 Baseline Models and Experimental Models **5**

5 Experimental results **6**

1 Baseline paper : Collaborative Filtering by Similarity Fusion

Our work is based on the paper *"Unifying User-based and Item-based Collaborative Filtering Approaches by Similarity Fusion"* by Jun Wang, Arjen P. de Vries and Marcel J. T. Reinders (SIGIR 2006). This section recalls its notation and probabilistic model.

1.1 Problem and Motivation

Let X be the $K \times M$ user–item matrix, where $x_{k,m}$ is the rating given by user u_k to item i_m, and $x_{k,m} = \emptyset$ when unknown. Memory-based collaborative filtering traditionally averages neighboring ratings in two isolated ways:

- **User-based (SUR):** Predicts $x_{k,m}$ using ratings that similar users gave to the target item i_m.
- **Item-based (SIR):** Predicts $x_{k,m}$ using ratings that the target user u_k gave to similar items.

Here $SUR = \{x_{a,m} \mid u_a \in \mathcal{S}_u(u_k)\}$ and $SIR = \{x_{k,b} \mid i_b \in \mathcal{S}_i(i_m)\}$ are the sets of ratings used by each approach, where $\mathcal{S}_u(u_k)$ contains the N users most similar to u_k that rated i_m, and $\mathcal{S}_i(i_m)$ the N items most similar to i_m that were rated by u_k. The user similarity s_u is the cosine between rating vectors and the item similarity s_i is the adjusted cosine (user mean subtracted).

Because matrices are sparse, these neighboring ratings are frequently missing. To overcome this, the authors fuse SUR and SIR, and introduce a third source, the **SUIR** set (similar user item ratings):

$$SUIR = \{x_{a,b} \mid u_a \in \mathcal{S}_u(u_k),\ i_b \in \mathcal{S}_i(i_m),\ a \neq k,\ b \neq m\}. \tag{1}$$

Although individually less accurate, SUIR ratings are much more numerous and act as a *background model* to smooth predictions.

1.2 Normalisation and Individual Predictors

Users rate differently (some avoid extreme values, others do not), and some items receive systematically higher ratings because of their audience. To account for this, each known rating $x_{a,b}$ is treated as an *individual predictor* of the test rating $x_{k,m}$, after correcting for user and item rating behaviour:

$$p_{k,m}(x_{a,b}) = x_{a,b} - (\bar{x}_a - \bar{x}_k) - (\bar{x}_b - \bar{x}_m), \tag{2}$$

where $\bar{x}_a$ and $\bar{x}_k$ are the mean ratings of users u_a and u_k, and $\bar{x}_b$ and $\bar{x}_m$ those of items i_b and i_m. This formula assumes that the test rating deviates from its expected value (global mean plus user and item effects) by the same amount as $x_{a,b}$ does. Appendix A of the paper shows that it is equivalent to the double centering $x_{a,b} - \bar{x}_a - \bar{x}_b + \bar{x}$ of the matrix (with $\bar{x}$ the global mean), which does not depend on whether users or items are centred first.

1.3 Probabilistic Fusion Framework

Collaborative filtering is cast as the estimation of $P(x_{k,m} \mid \mathcal{P}_{k,m})$, the probability of the unknown rating given the pool of individual predictors. Two hidden binary variables control which sources are used:

- I_1: $I_1 = 1$ means that the rating depends only on SUR, and $I_1 = 0$ only on SIR. We write $\lambda = P(I_1 = 1)$;
- I_2: $I_2 = 1$ means that the rating depends only on SUIR, and $I_2 = 0$ on SUR and SIR. We write $\delta = P(I_2 = 1)$.

Marginalising over I_1 and I_2 gives

$$\begin{aligned} P(x_{k,m} \mid SUR, SIR, SUIR) = {} & \big[\lambda\, P(x_{k,m} \mid SUR) + (1-\lambda)\, P(x_{k,m} \mid SIR)\big](1-\delta) \\ & + \delta\, P(x_{k,m} \mid SUIR). \end{aligned} \tag{3}$$

The authors combine the sources with a *sum rule* rather than a product rule: the product, which follows from an independence assumption, is considered too sensitive to estimation errors. The prediction $\hat{x}_{k,m}$ is the expected value of $x_{k,m}$ under this distribution.

1.4 Probability Estimation and Combined Similarity

The probability that a predictor $x_{a,b}$ is "right" is assumed proportional to its similarity to the test rating. For SUR it is proportional to $s_u(u_k, u_a)$, for SIR to $s_i(i_m, i_b)$. For instance:

$$P(x_{k,m} = r \mid SUR) = \frac{\sum_{x_{a,b} \in SUR,\ p_{k,m}(x_{a,b}) = r} s_u(u_k, u_a)}{\sum_{x_{a,b} \in SUR} s_u(u_k, u_a)}, \tag{4}$$

and similarly for SIR. For a rating of SUIR, which depends on both a user and an item, the two similarities are combined in a Euclidean dissimilarity space:

$$s_{ui}(x_{k,m}, x_{a,b}) = \frac{1}{\sqrt{\left(\frac{1}{s_u(u_k, u_a)}\right)^2 + \left(\frac{1}{s_i(i_m, i_b)}\right)^2}}. \tag{5}$$

This ensures that only ratings at the core of the fusion square (users who are highly similar and items that are highly similar) receive significant weight.

1.5 Unified Weights and Final Prediction

The final prediction $\hat{x}_{k,m}$ is a combination of all normalized individual predictors:

$$\hat{x}_{k,m} = \sum_{x_{a,b}} p_{k,m}(x_{a,b})\, W_{k,m}^{a,b}, \quad \text{with} \quad \sum_{x_{a,b}} W_{k,m}^{a,b} = 1, \tag{6}$$

where the unified weights $W_{k,m}^{a,b}$ combine user similarities (s_u), item similarities (s_i), and their Euclidean combination (s_{ui} for SUIR) scaled by two interpolation parameters:

- λ: Weighs SUR vs. SIR ($\lambda = 1$ is pure user-based, $\lambda = 0$ is pure item-based).
- δ: Controls the influence of the 2D background model (SUIR). When data is extremely sparse, a high δ (≈ 0.8) leverages the background model to smooth predictions and drastically reduce error (MAE/RMSE).

2 Our take

Before discovering this paper, we had already tried several KNN approaches, and we wanted to integrate our best ideas into the paper's model.

2.1 Integrating the movie features into our model

The paper did not have access to that information, and we wanted to find a way to add it to the model in a controllable way, so that it could be fused with the rating-based similarity. We consider two pieces of information about our movies: their genres and their release year. For the first one, s_{genre}, we used a Jaccard similarity as the natural choice, since genres form a set. For the year, we ended up using an exponential decay similarity, as it guarantees values in $[0, 1]$, is decreasing, and is easily parameterisable through a single bandwidth τ:

$$s_{\text{year}}(i, j) \;=\; \exp\!\left(-\frac{|y_i - y_j|}{\tau}\right),$$

where y_i is the release year of movie i. We combine the two aspects as a weighted sum controlled by a parameter w_{genre}, and we then blend the result with the rating-based similarity through a parameter α:

$$s_{\text{content}}(i, j) = w_{\text{genre}}\, s_{\text{genre}}(i, j) + (1 - w_{\text{genre}})\, s_{\text{year}}(i, j),$$

$$s_{\text{i}}(i, j) = \alpha\, s_{\text{rating}}(i, j) + (1 - \alpha)\, s_{\text{content}}(i, j).$$

An important note: the two similarities do not live on the same scale at all. The content-based one averages around 0.2, while the rating-based one is closer to 0.003, since two movies share very few raters in a sparse matrix. The best α we found was 0.3; because of this scale gap, the value of α alone is not a reliable indication of how much each source actually contributes because it seems that the feature is the most important part, but the RMSE gains are quite small; they are around 0.01.

2.2 Computing the neighborhood

In the paper, a single parameter N determines the number of neighbors considered, both in the movie space and in the user space. As our matrix is far from square, we found it more relevant to use two separate parameters, k_{user} and k_{movie}. Moreover, the paper looks for the N nearest *ratings* of the target, i.e. the N most similar users *among those who rated the target movie*. For a movie with few ratings, this means we may end up including users with a very low similarity, and the computation is expensive, since the set of neighbors has to be rebuilt for every single rating.

To obtain something more consistent and cheaper to compute, we decided to fix the number of neighbors instead so that each user's and each movie's neighborhood is computed once and reused for all ratings. This guarantees that we only take into account users or movies that are genuinely close to the target. The price of this choice is the uncertainty on the number of

ratings actually involved: if k is set too low, no neighbor may have rated the target, and the prediction falls back on the baseline. The best values we found were $k_{\text{user}} = 400$ and $k_{\text{movie}} = 800$, reducing the RMSE by 0.05 in our cross-validation.

Since the direct neighborhoods turned out to be quite large, the rectangle used for the fusion became very large as well, and it made little sense to us to keep that many cells, most of which are noise. To limit this effect, we introduced two additional parameters, $cross_{\text{user}}$ and $cross_{\text{movie}}$, which bound the rectangle in both directions, shaving another 0.05 off the RMSE.

2.3 Computing the similarity

Computing the similarity between items is the most important part of a KNN approach, and we experimented with it quite a bit. We tried a cosine similarity on the residuals from the baseline rather than on the raw ratings. The paper mentions this idea but only applies it to the movie similarity, where the user's average rating is subtracted from each co-rated pair. Our variant first looked promising, but after further investigation we realized that it substantially increased the RMSE, performing worse than the paper's original formulation. We therefore kept the similarity as defined in the paper.

2.4 Computing the similarity inside the rectangle

The novel idea of this paper is to exploit the ratings given by similar users to similar movies, which yields a rectangle of ratings to aggregate. Each cell must be weighted by a combination of the user similarity s_{u} and the movie similarity s_{i}. We tested the approach described in the paper, which combines them in a Euclidean dissimilarity space as well as a much simpler alternative, the plain product of the two similarities:

$$s_{\text{ui}}\big(x_{k,m}, x_{a,b}\big) \;=\; s_{\text{u}}(u_k, u_a) \cdot s_{\text{i}}(i_m, i_b).$$

The product form did not help at this stage and even slightly increased the RMSE, but it is far cheaper to compute: unlike the Euclidean combination, it factorises, so the whole rectangle can be evaluated as a sequence of matrix products instead of cell by cell.

2.5 Adding regularization

We identified a weakness in the paper's normalization. Each rating is centered using the raw average of its user and of its movie, and those averages are estimated from very little data. A movie rated twice with five stars gets the same treatment as a movie rated three hundred times, even though two ratings prove almost nothing. With close to ten thousand movies, most of which have a handful of ratings, this noise propagates into every prediction and into every residual.

We therefore replaced the raw averages by regularized biases, estimated jointly as the solution of a penalized least-squares problem:

$$\min_{b_{\text{u}},\, b_{\text{i}}} \sum_{(u,i)\,\in\,\mathcal{K}} \big(r_{ui} - \mu - b_u - b_i\big)^2 \;+\; \beta_{\text{u}} \sum_u b_u^2 \;+\; \beta_{\text{i}} \sum_i b_i^2,$$

where $\mathcal{K}$ is the set of known ratings and μ the global mean. The two penalty terms are controlled by β_{u} and β_{i} (`reg_u` and `reg_i` in our implementation). Holding one side fixed, the optimal bias of the other has a closed form, for instance for a movie:

$$b_i \;=\; \frac{1}{\beta_{\text{i}} + n_i} \sum_{u\,:\,(u,i)\in\mathcal{K}} \big(r_{ui} - \mu - b_u\big) \;=\; \frac{n_i}{n_i + \beta_{\text{i}}} \times \overline{\text{residu}}_i$$

with n_i the number of ratings of movie i. The observed average residual is thus multiplied by a confidence factor between 0 and 1: with $\beta_{\text{i}} = 10$, a movie rated twice keeps only 17% of its apparent bias, one rated ten times keeps half of it, and one rated three hundred times keeps 97%. Rare movies are shrunk towards the global mean, popular ones are left essentially untouched. We use two separate parameters because the two sides carry very different amounts of rates.

The two biases depend on each other: a movie watched mostly by harsh raters looks bad until the user biases are known, and a user who only watched poor movies looks harsh until the movie biases are known. We therefore solve the problem by alternating least squares, recomputing b_{i} with b_{u} fixed and then the converse, over `n_iter` passes. Each half-step solves its subproblem exactly and decreases the objective, and since the problem is convex the procedure converges quickly; fifteen passes were enough in all our experiments.

This regularized baseline serves two purposes in the model: it provides the value on top of which the neighborhood deviation is added, and it defines the residual matrix from which the deviations themselves are computed.

2.6 Predicting a missing rating

We can now put all the pieces together. To predict the rating of user u for movie i, we start from the regularised baseline and add a deviation estimated from three sources of neighboring ratings:

$$\hat{r}_{ui} \;=\; b_{ui} \;+\; \lambda(1-\delta)\, e_{\text{U}}(u,i) \;+\; (1-\lambda)(1-\delta)\, e_{\text{I}}(u,i) \;+\; \delta\, e_{\text{X}}(u,i)$$

$$b_{ui} = \mu + b_u + b_i$$

where the prediction is clipped to the valid rating range $[0.5, 5]$. Let $\mathcal{S}_{\mathrm{u}}(u)$ denote the k_{user} nearest users of u, $\mathcal{S}_{\mathrm{i}}(i)$ the k_{movie} nearest movies of i, and let $c_{vj} = r_{vj} - b_{vj}$ be the residual of a known rating. Writing $m_{vj} = 1$ if the rating (v, j) is known and 0 otherwise, the three deviations are weighted averages of residuals:

$$e_{\mathrm{U}}(u,i) = \frac{\sum_{v \in \mathcal{S}_{\mathrm{u}}(u)} s_{\mathrm{u}}(u,v)\, m_{vi}\, c_{vi}}{\sum_{v \in \mathcal{S}_{\mathrm{u}}(u)} s_{\mathrm{u}}(u,v)\, m_{vi}}, \qquad e_{\mathrm{I}}(u,i) = \frac{\sum_{j \in \mathcal{S}_{\mathrm{i}}(i)} s_{\mathrm{i}}(i,j)\, m_{uj}\, c_{uj}}{\sum_{j \in \mathcal{S}_{\mathrm{i}}(i)} s_{\mathrm{i}}(i,j)\, m_{uj}},$$

$$e_{\mathrm{X}}(u,i) = \frac{\sum_{v \in \mathcal{C}_{\mathrm{u}}(u)} \sum_{j \in \mathcal{C}_{\mathrm{i}}(i)} s_{\mathrm{u}}(u,v)\, s_{\mathrm{i}}(i,j)\, m_{vj}\, c_{vj}}{\sum_{v \in \mathcal{C}_{\mathrm{u}}(u)} \sum_{j \in \mathcal{C}_{\mathrm{i}}(i)} s_{\mathrm{u}}(u,v)\, s_{\mathrm{i}}(i,j)\, m_{vj}},$$

with $\mathcal{C}_{\mathrm{u}}(u)$ and $\mathcal{C}_{\mathrm{i}}(i)$ the $cross_{\mathrm{user}}$ and $cross_{\mathrm{movie}}$ nearest neighbors, i.e. the truncated neighborhoods used for the rectangle. The masks m_{vj} make the missing cells vanish from both the numerator and the denominator, so each source is normalised by the weight it actually used. When a source has no usable rating at all, its deviation is zero and the prediction falls back on the baseline for that part.

Written this way, the three terms differ only in which ratings they aggregate: the same movie rated by similar users, other movies rated by the same user, and similar movies rated by similar users. Note that e_{X} uses the product of the two similarities rather than the Euclidean combination of the paper. Once the regularised baseline is in place, the product became slightly better than the Euclidean form, which reversed the ranking we had observed before regularisation, and it has the practical advantage of factorising into matrix products.

Our final model uses $k_{\mathrm{user}} = 500$ and $k_{\mathrm{movie}} = 900$ for the direct neighborhoods, restricted to $cross_{\mathrm{user}} = 100$ and $cross_{\mathrm{movie}} = 400$ inside the rectangle. Similarities follow the paper, with the content-based movie similarity blended in at $\alpha = 0.3$. The fusion weights are $\lambda = 0.4$, slightly favouring the movie side, and $\delta = 0.1$, which gives the rectangle a modest but non-zero contribution. The baseline is regularised with $\beta_{\mathrm{u}} = \beta_{\mathrm{i}} = 10$ over five alternating passes. All hyperparameters were selected by cross-validation on the training matrix, and the test set was used only once, to produce the final figure reported below.

3 Experimental setup

Throughout our research, we followed a consistent and reproducible methodology. To train and evaluate our models, we combined the two datasets provided on the website and reconstructed a single dataset, which we then split into three subsets: a training set (80% of the data), a validation set (10% of the data), and a test set (10% of the data). The dataset reconstruction consisted of randomly assigning the data points to the three subsets. To ensure consistency and reproducibility, we fixed a random seed for each dataset reconstruction. This ensured that the same data splits were used across all models and allowed us to rerun the experiments whenever updates were made to the models. For each model, we conducted 30 independent experiments to obtain more reliable performance estimates and enable a fair comparison between models under consistent conditions.

4 Baseline Models and Experimental Models

To evaluate the contribution of each modification and assess its impact on model performance, we present four successive variants of our KNN-based model. Each variant introduces specific improvements to the original approach, allowing us to analyze the contribution of individual design choices.

- **Paper KNN**: Implements the model proposed in the original paper without any modifications. It serves as the reference point for evaluating the improvements introduced in our subsequent variants.
- **Movie Feature KNN**: Extends the original model by incorporating movie features, which are not considered in the original approach. This variant allows us to evaluate the contribution of additional movie-related information to recommendation performance.
- **K-Cross KNN**: Introduces additional parameters, as explained in Section 2.2, to compute similarities over a rectangular neighborhood rather than restricting the computation to a square one. This modification provides greater flexibility in the selection of neighboring users and items.
- **Final KNN**: Combines the improvements introduced in the previous variants and incorporates additional regularization parameters, as described in Section 2.5. This final configuration aims to improve recommendation performance while controlling overfitting.

In addition to these variants, we compare our results against three collaborative filtering baselines, covering both simple averaging and latent factor-based approaches, as well as neural methods:

- **User Mean**: Predicts ratings based on the average rating given by each user. This baseline captures individual rating tendencies without modeling user–item interactions explicitly.
- **Alternating Least Squares (ALS)**: Learns latent representations of users and items by alternately optimizing their respective factor matrices. These representations are used to estimate user preferences and predict ratings.
- **Neural Collaborative Filtering (https://arxiv.org/abs/1708.05031)**: Uses neural networks to model potentially non-linear user–item interactions, providing a more flexible alternative to traditional collaborative filtering methods.

5 Experimental results

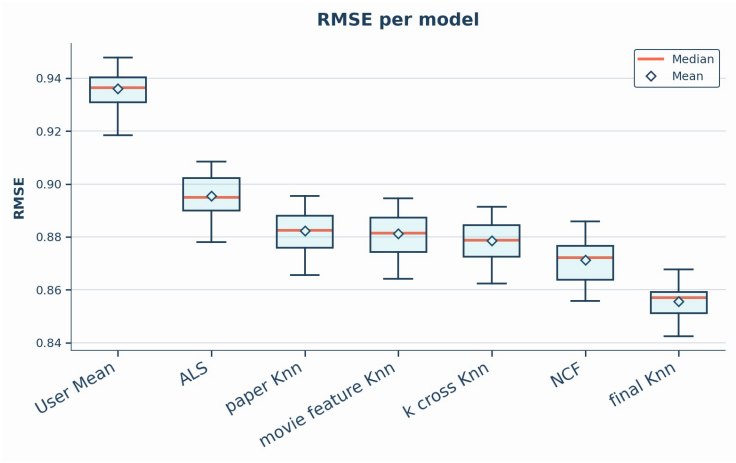


Figure 1: Comparison of RMSE scores

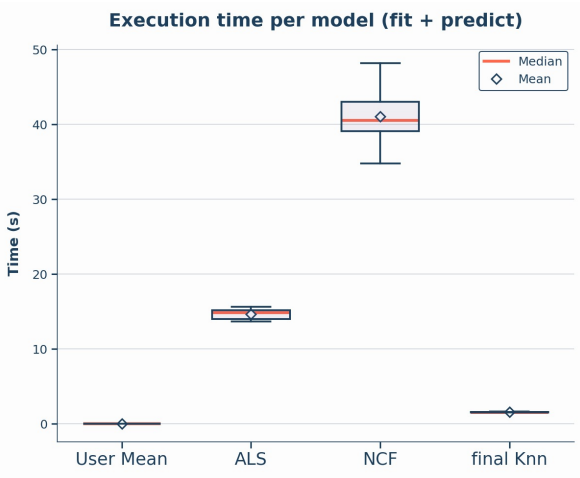


Figure 2: Comparison of execution times

Figure 1 summarizes the performance of the different models in terms of RMSE. We observe that both movie feature Knn and k cross Knn slightly outperform paper KNN, achieving mean RMSE values of 0.8812 and 0.8785, respectively, compared with 0.8823 for paper Knn. These results suggest that incorporating movie features and introducing a more flexible neighborhood structure provide modest improvements over the original model. However, the most substantial performance gain is achieved by the final Knn model, which incorporates regularization parameters in addition to the previous improvements. This modification significantly reduces the mean RMSE by approximately 0.027 compared with the preceding variants, allowing our model to outperform the NCF baseline. This highlights the importance of regularization in improving the predictive accuracy of our approach. Regarding the other baselines, both User Mean and ALS are outperformed by Paper KNN, indicating that even the original KNN-based approach provides more accurate predictions than these methods on our dataset. Finally, the final Knn model also exhibits lower variability in its RMSE scores, with a standard deviation of approximately 0.0070, compared with around 0.0080 for the other variants. This suggests that the final model delivers more consistent performance across the evaluated samples or experimental runs.

Figure 2 compares the execution times of the different models on the training and test sets. One notable observation is the computational cost of ALS and NCF, which require an average of approximately 14 and 41 seconds, respectively, compared with less than 2 seconds for our final Knn model.

This significant reduction in execution time highlights the computational efficiency of our approach. In particular, optimizations such as NumPy vectorization help reduce computational overhead by enabling efficient operations on entire arrays. Consequently, our final Knn model achieves competitive predictive performance while requiring substantially less computation time than ALS and NCF. These results demonstrate that our approach offers an effective balance between recommendation accuracy and computational efficiency.

To conclude, the results presented in this section demonstrate that the successive improvements to our Knn model have enabled us to achieve a good trade-off between predictive accuracy and execution time. However, we believe that we have explored most of the promising improvements within the scope of our current approach. Further gains in predictive performance might be achieved using more advanced methods, particularly deep learning-based models, which can capture more complex user-item interactions. Nevertheless, these approaches generally require greater computational resources and longer execution times. The main advantage of our final Knn model therefore lies in its computational efficiency, making it a competitive alternative when fast predictions and low computational overhead are key requirements.